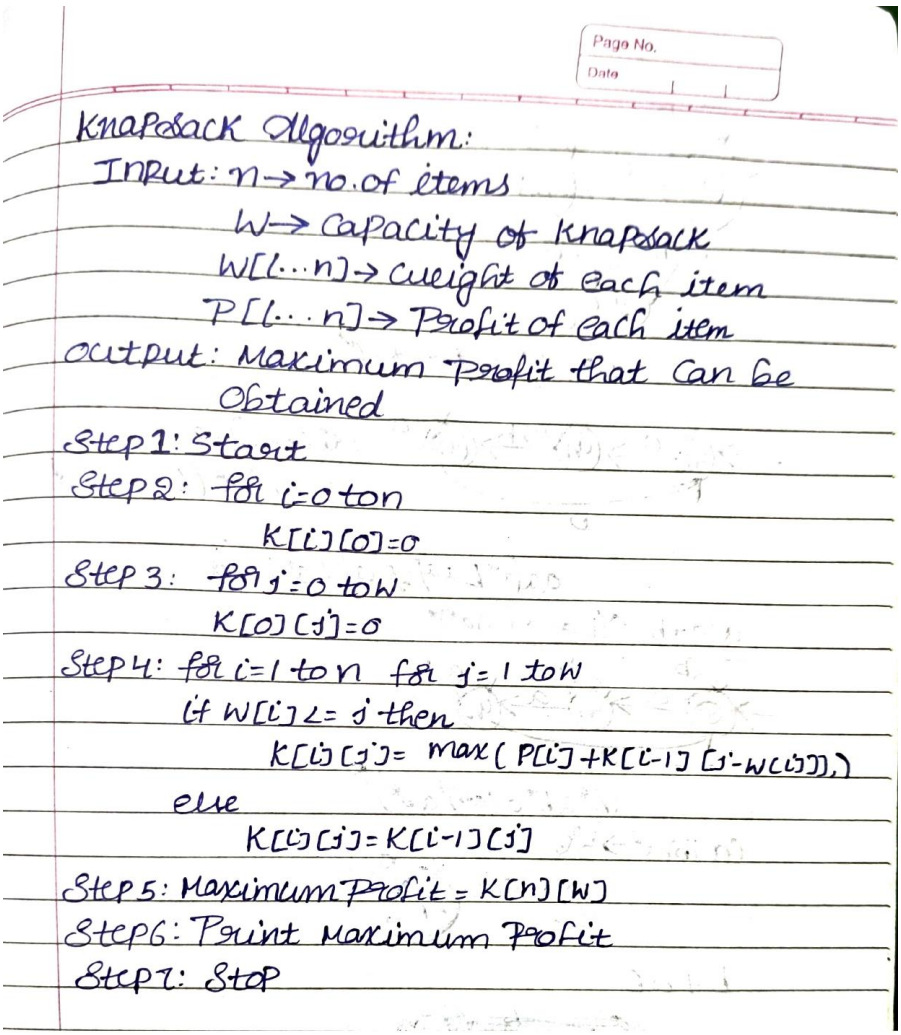
PRACTICAL-5

Aim: To Implementation of a knapsack problem using dynamic programming

Algorithm:



Knapsack Algorithm:
Input: n → no. of items
W → capacity of knapsack
W[1...n] → weight of each item
P[1...n] → Profit of each item
output: Maximum Profit that can be obtained
Step 1: Start
Step 2: for i=0 to n
K[i][0]=0
Step 3: for j=0 to W
K[0][j]=0
Step 4: for i=1 to n for j=1 to W
if W[i]<= j then
K[i][j]= max(P[i]+K[i-1][j-W[i]],)
else
K[i][j]=K[i-1][j]
Step 5: Maximum Profit = K[n][W]
Step 6: Print Maximum Profit
Step 7: Stop

Code:

```
#include<iostream>

using namespace std;

 class Knapsack {

 private:

   int w[10];       // weights

   int v[10];       // values

   int n;           // number of items

   int W;           // capacity
```

```
    int dp[10][100]; // dp table

public:

    // ── Get Input ──
    void getInput() {
        cout << "Enter number of items   : ";
        cin  >> n;
        cout << "Enter knapsack capacity : ";
        cin  >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << " weight and value : ";
            cin  >> w[i] >> v[i];
        }
    }

    // ── Build dp table ──
    void solve() {

        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++) dp[i][0] = 0;
        for(int j = 0; j <= W; j++) dp[0][j] = 0;

        // fill table
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= W; j++) {

                if(w[i] <= j) {
                    // take or not take
                    dp[i][j] = max(
```

```
                v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]
            );
        } else {
            // cannot take item i
            dp[i][j] = dp[i-1][j];
        }
    }
  }
}

// ── Print dp table ──
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "     ";
    for(int j = 0; j <= W; j++)
        cout << j << "  ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << "  ";
        cout << endl;
    }
}

// ── Find selected items ──
void findItems() {
```

```
        cout << "\nSelected Items:" << endl;
        int j = W;
        for(int i = n; i >= 1; i--) {
            if(dp[i][j] != dp[i-1][j]) {
                cout << "Item " << i
                    << " (w=" << w[i]
                    << ", v=" << v[i]
                    << ") TAKEN " << endl;
                j = j - w[i];
            } else {
                cout << "Item " << i
                    << " (w=" << w[i]
                    << ", v=" << v[i]
                    << ") NOT taken " << endl;
            }
        }
    }

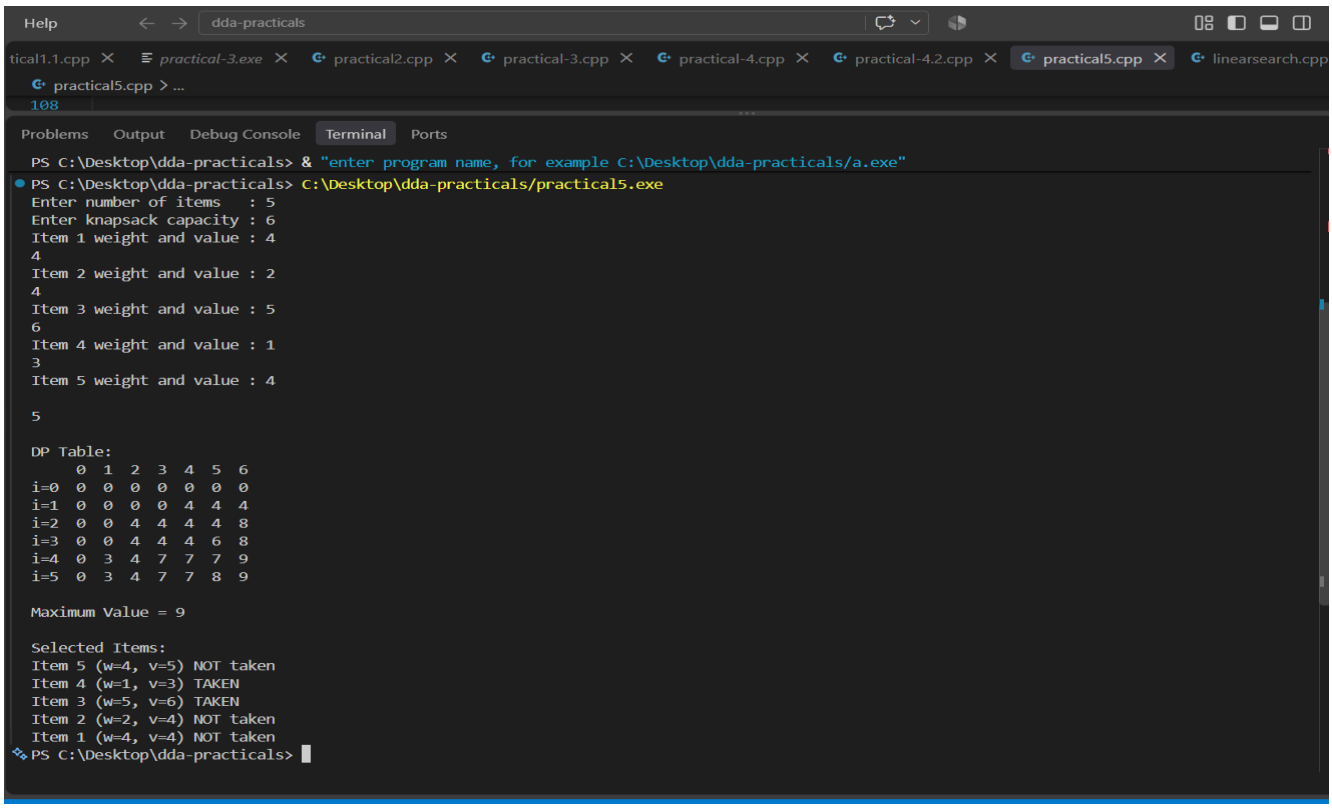
    // ── Show result ──
    void showResult() {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};

// ── MAIN ──
int main() {

    Knapsack k;
```

```
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

Output:



Time complexity:

Marwadi University
Marwadi Chandarana Group

NAAC A+

Time Complexity:

for i = 1 to n ⟶ n

for j = 1 to W ⟶ W

If W[i] <= j ⟶ 1

K[i][j] = max(P[i] + K[i-1][j-W[i]]) ⟶ 1

else

K[i][j] = K[i-1][j] ⟶ 1

So,

T = n x W x O(1)

Ignoring constant:

T = nW

Suppose:

n = 4 items, W = 5 capacity

no. of table calculations:

n x W => 4 x 5 = 20

So, the algorithm performs 20 main iterations

1. Worst case:

every table cell is calculated

T worst = nxw

if n = 4, w = 5

T worst = 4 x 5 = 20

Worst case = O(nW)

2. Average case: even in average case also the algorithm still goes through all n x W table positions.

T average = nxw

for n = 4, W = 5:

T average = 4 x 5 = 20

Average case = O(nW)

Result: The 0/1 Knapsack algorithm finds the combination of items having maximum profit without exceeding the given knapsack capacity.